Convolutional Neural Networks

DTSC411

Why Convolutional Neural Networks?

Problems with Fully Connected Networks for Images:

- **Too many parameters:** $224 \times 224 \times 3$ image $= 150{,}528$ inputs
  - Single hidden layer with 1000 neurons $=$ 150M parameters!
- **No spatial structure:** Treats pixels as independent features
- **Not translation invariant:** Cat in different position $=$ different features
- **Overfitting:** Too many parameters, not enough data

CNN Solutions:

- **Local connectivity:** Each neuron connects to small region
- **Parameter sharing:** Same filter applied everywhere
- **Spatial hierarchy:** Learn features from simple to complex
- **Translation invariance:** Same filter detects feature anywhere

Result: Dramatically fewer parameters, better generalization

Convolution Operation

2D Convolution: Slide filter over image, compute dot product

$$Y[i,j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X[i+m, j+n] \cdot K[m,n]$$

where K is the kernel (filter), k is kernel size

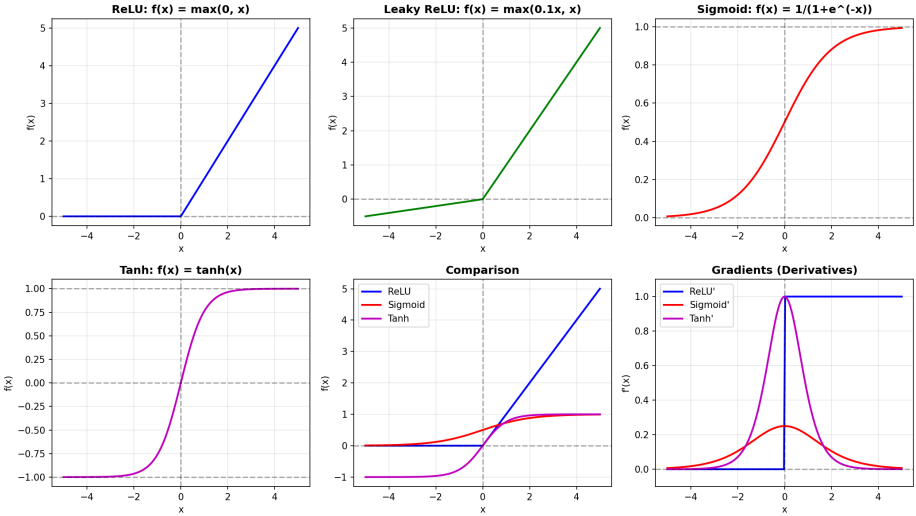
Properties:

- Filter detects specific pattern
- Learned during training (not hand-crafted)
- Same filter applied everywhere
- Multiple filters = multiple features

Output Size:

$$\text{output_size} = \frac{\text{input_size} - \text{kernel_size} + 2 \times \text{padding}}{\text{stride}} + 1$$

Activation Functions



Activation Functions

1. **ReLU (Rectified Linear Unit):** Most common

$$f(x) = \max(0, x)$$

Advantages: Fast, no vanishing gradient, sparse activation

2. **Leaky ReLU:** Allows small negative values

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{otherwise} \end{cases}$$

where $\alpha \approx 0.01$

3. **Sigmoid:** Rarely used in hidden layers

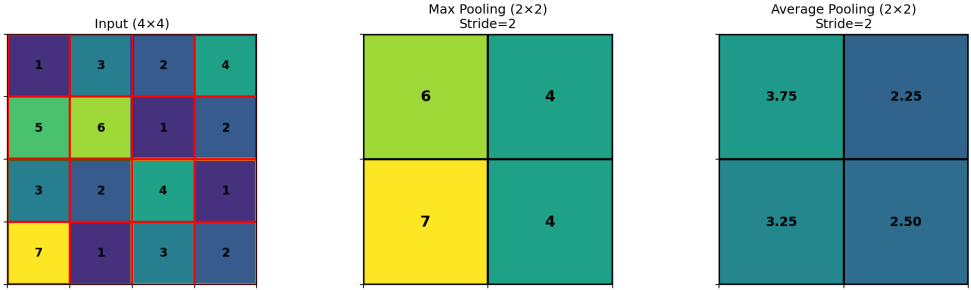
$$f(x) = \frac{1}{1 + e^{-x}}$$

Problem: Vanishing gradient

4. **Tanh:** Zero-centered sigmoid

$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Pooling Layers



Max Pooling: Takes maximum value in each region (preserves strong features)
Avg Pooling: Takes average value (smoother downsampling)

Pooling Layers: Details

Purpose: Downsample spatial dimensions, reduce parameters

Max Pooling: Take maximum in each region

- Most common: 2×2 with stride 2
- Reduces spatial size by 2x
- Preserves most important features
- Provides translation invariance

Average Pooling: Take average in each region

- Smoother downsampling
- Less common in hidden layers
- Often used before final classification (Global Average Pooling)

Global Average Pooling (GAP):

- Reduce entire feature map to single value per channel
- Replaces fully connected layers
- Reduces overfitting
- Used in modern architectures

Standard CNN Architecture

Typical Pattern:

$$\text{INPUT} \rightarrow [\text{CONV} \rightarrow \text{RELU} \rightarrow \text{POOL}] \times N \rightarrow \text{FC} \rightarrow \text{OUTPUT}$$

Common Design Principles:

1. **Spatial dimensions decrease:** Pooling or strided convolutions
2. **Number of channels increases:** More abstract features
3. **Multiple conv layers before pooling:** VGG pattern
4. **Final layers:** Fully connected or Global Average Pooling

Example Flow:

- Input: $224 \times 224 \times 3$
- After CONV1 + POOL1: $112 \times 112 \times 64$
- After CONV2 + POOL2: $56 \times 56 \times 128$
- After CONV3 + POOL3: $28 \times 28 \times 256$
- After CONV4 + POOL4: $14 \times 14 \times 512$
- After CONV5 + POOL5: $7 \times 7 \times 512$
- Flatten: $7 \times 7 \times 512 = 25088$
- FC layers: $25088 \rightarrow 4096 \rightarrow 4096 \rightarrow 1000$

CNN Architecture Evolution

Evolution: From shallow (LeNet) to deep (ResNet, EfficientNet)

Key innovations:

- ReLU
- Dropout
- BatchNorm
- Skip Connections

LeNet-5 (1998)

Yann LeCun - First successful CNN for digit recognition

Architecture:

1. INPUT: 32×32 grayscale image
2. C1: Conv 5×5, 6 filters $\rightarrow 28 \times 28 \times 6$
3. S2: Average pooling $2 \times 2 \rightarrow 14 \times 14 \times 6$
4. C3: Conv 5×5, 16 filters $\rightarrow 10 \times 10 \times 16$
5. S4: Average pooling $2 \times 2 \rightarrow 5 \times 5 \times 16$
6. C5: Conv 5×5, 120 filters $\rightarrow 1 \times 1 \times 120$
7. F6: Fully connected, 84 neurons
8. OUTPUT: 10 classes (digits 0-9)

Key Innovations:

- Demonstrated CNNs work for practical applications
- Used on bank checks in 1990s
- Established Conv $\rightarrow$ Pool pattern

Parameters: 60K (tiny by today's standards)

AlexNet (2012) - The Breakthrough

Krizhevsky, Sutskever, Hinton - Won ImageNet 2012
Architecture:

1. INPUT: $227 \times 227 \times 3$
2. CONV1: 11×11, stride 4, 96 filters $\rightarrow 55 \times 55 \times 96$
3. POOL1: 3×3, stride 2 $\rightarrow 27 \times 27 \times 96$
4. CONV2: 5×5, 256 filters $\rightarrow 27 \times 27 \times 256$
5. POOL2: 3×3, stride 2 $\rightarrow 13 \times 13 \times 256$
6. CONV3: 3×3, 384 filters $\rightarrow 13 \times 13 \times 384$
7. CONV4: 3×3, 384 filters $\rightarrow 13 \times 13 \times 384$
8. CONV5: 3×3, 256 filters $\rightarrow 13 \times 13 \times 256$
9. POOL5: 3×3, stride 2 $\rightarrow 6 \times 6 \times 256$
10. FC6: 4096 neurons, FC7: 4096 neurons, FC8: 1000 classes

Parameters: 60M (mostly in FC layers)
Top-5 Error: 15.3% (vs 26% for second place)

AlexNet: Key Innovations

What Made AlexNet Revolutionary:

1. **ReLU Activation:** Instead of tanh/sigmoid
   - 6x faster training
   - No vanishing gradient problem
2. **Dropout:** 0.5 dropout in FC layers
   - Prevents overfitting
   - Ensemble effect
3. **Data Augmentation:**
   - Random crops and flips
   - PCA color augmentation
4. **GPU Training:** Split across 2 GPUs
   - Made training feasible
   - Parallelization strategy
5. **Local Response Normalization (LRN):**
   - Lateral inhibition
   - Later replaced by Batch Normalization

Impact: Sparked the deep learning revolution in computer vision

VGG (2014)

Simonyan & Zisserman - Very Deep Networks
Key Idea: Depth matters, use small filters

Design Principles:

- **Small filters:** Only 3×3 convolutions
- **Deep network:** 16-19 layers (VGG-16, VGG-19)
- **Simple architecture:** Easy to understand and modify
- **2x** 3×3 **conv** $=$ 5×5 **receptive field**
- **3x** 3×3 **conv** $=$ 7×7 **receptive field**
- But fewer parameters and more non-linearity!

VGG-16 Architecture:

- 13 conv layers (all 3×3)
- 5 max pooling layers (2×2)
- 3 FC layers (4096, 4096, 1000)
- Pattern: [CONV-CONV-POOL] repeated

VGG-16 Detailed Architecture

Layer-by-layer:

Block	Layers	Output Size	Channels
Input	-	224×224	3
Block 1	2x CONV3-64 + POOL	112×112	64
Block 2	2x CONV3-128 + POOL	56×56	128
Block 3	3x CONV3-256 + POOL	28×28	256
Block 4	3x CONV3-512 + POOL	14×14	512
Block 5	3x CONV3-512 + POOL	7×7	512
FC	FC-4096, FC-4096, FC-1000	-	-

Advantages:

- Simple, uniform architecture
- Strong performance
- Good for transfer learning

Disadvantages:

- Very large model (138M parameters)
- Slow to train
- Most parameters in FC layers

ResNet (2015) - Residual Networks

He et al. - Revolutionized Deep Learning

Problem: Very deep networks hard to train

- Degradation problem: Deeper networks perform worse
- NOT due to overfitting (training error also higher)
- Vanishing/exploding gradients

Solution: Residual Connections (Skip Connections)

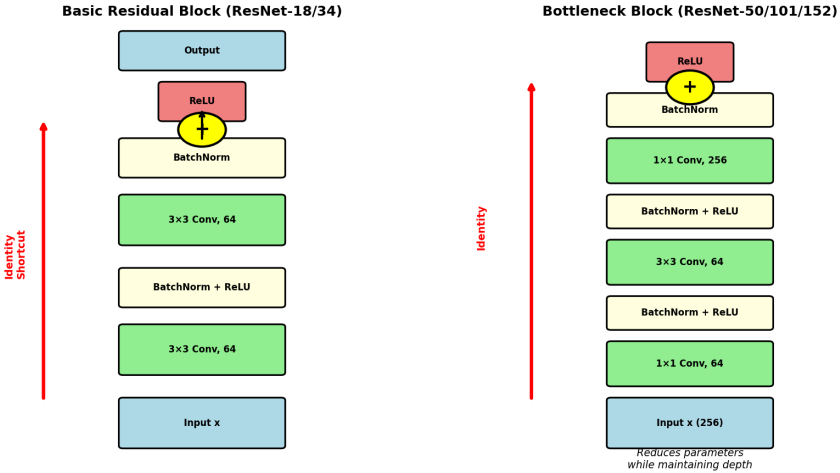
$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$$

Instead of learning $\mathbf{y} = \mathcal{F}(\mathbf{x})$, learn residual $\mathcal{F}(\mathbf{x}) = \mathbf{y} - \mathbf{x}$

Why This Works:

- Easier to learn identity mapping: just set $\mathcal{F}(\mathbf{x}) = 0$
- Gradients flow directly through skip connections
- Network can be arbitrarily deep
- If layer doesn't help, it learns to pass input through

ResNet: Residual Block



Key idea: $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$ (learn residual)

ResNet: Residual Block Details

Basic Residual Block:

1. Input $\mathbf{x}$
2. CONV 3×3, BatchNorm, ReLU
3. CONV 3×3, BatchNorm
4. Add input: $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$
5. ReLU activation

Bottleneck Block (ResNet-50+):

1. CONV 1×1 (reduce dimensions)
2. CONV 3×3 (process)
3. CONV 1×1 (restore dimensions)
4. Add input

Reduces parameters while maintaining performance

Common Variants:

- ResNet-18, ResNet-34: Basic blocks
- ResNet-50, ResNet-101, ResNet-152: Bottleneck blocks

ResNet-50 Architecture

Stage	Output Size	Blocks	Channels
Conv1	112×112	1	64
Conv2_x	56×56	3	64, 64, 256
Conv3_x	28×28	4	128, 128, 512
Conv4_x	14×14	6	256, 256, 1024
Conv5_x	7×7	3	512, 512, 2048
Global Avg Pool	1×1	-	2048
FC	-	-	1000

Parameters: 25.6M (much less than VGG despite being deeper!)

Performance:

- Top-5 Error: 3.57% on ImageNet
- Won ImageNet 2015
- Enabled training networks with 100+ layers

EfficientNet (2019)

Tan & Le - Compound Scaling

Key Idea: Scale depth, width, and resolution together

Traditional Scaling (one dimension):

- **Depth:** Add more layers (ResNet-50 $\rightarrow$ ResNet-101)
- **Width:** More channels per layer
- **Resolution:** Larger input images

Compound Scaling (all together):

$$\text{depth: } d = \alpha^\phi$$

$$\text{width: } w = \beta^\phi$$

$$\text{resolution: } r = \gamma^\phi$$

subject to $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ and $\alpha \geq 1, \beta \geq 1, \gamma \geq 1$

where ϕ is user-specified compound coefficient

Result: Better accuracy with fewer parameters and FLOPs

EfficientNet Family

Model	Params	FLOPs	Top-1 Acc	Resolution
EfficientNet-B0	5.3M	0.39B	77.1%	224^2
EfficientNet-B1	7.8M	0.70B	79.1%	240^2
EfficientNet-B2	9.2M	1.0B	80.1%	260^2
EfficientNet-B3	12M	1.8B	81.6%	300^2
EfficientNet-B4	19M	4.2B	82.9%	380^2
EfficientNet-B5	30M	9.9B	83.6%	456^2
EfficientNet-B6	43M	19B	84.0%	528^2
EfficientNet-B7	66M	37B	84.3%	600^2

Key Features:

- Mobile Inverted Bottleneck (MBConv) blocks
- Squeeze-and-Excitation (SE) attention
- Best accuracy vs efficiency trade-off
- State-of-art on ImageNet (2019-2020)

CNN Architecture Comparison

Model	Year	Params	Depth	Top-5 Error
LeNet-5	1998	60K	5	-
AlexNet	2012	60M	8	15.3%
VGG-16	2014	138M	16	7.3%
ResNet-50	2015	25.6M	50	3.57%
EfficientNet-B0	2019	5.3M	-	2.9%

Evolution Trends:

- Deeper networks (5 $\rightarrow$ 50+ layers)
- More efficient (ResNet ¡ VGG in params despite deeper)
- Skip connections enable depth (ResNet)
- Attention mechanisms (EfficientNet)
- Better optimization (BatchNorm, better activations)

Current State (2023-2024):

- Vision Transformers (ViT) competitive
- ConvNeXt (modernized CNNs)

Loss Functions for Classification

1. **Cross-Entropy Loss (Multi-class):**

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{ic}\log(\hat{y}_{ic})$$

where y_{ic} is one-hot encoded label, $\hat{y}_{ic}$ is predicted probability

2. **Binary Cross-Entropy (Two classes):**

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}[y_i\log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

3. **Focal Loss (For class imbalance):**

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\alpha_i(1-\hat{y}_i)^{\gamma}\log(\hat{y}_i)$$

where γ focuses on hard examples (typically 2)

4. **Label Smoothing:**

$$y_{ic}^{\text{smooth}} = (1-\epsilon)y_{ic} + \frac{\epsilon}{C}$$

Optimization Algorithms

Goal: Minimize loss $\mathcal{L}(\theta)$ by updating parameters θ

1. Stochastic Gradient Descent (SGD):

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t)$$

Simple but can be slow and get stuck

2. SGD with Momentum:

$$v_{t+1} = \beta v_t + \nabla_\theta \mathcal{L}(\theta_t)$$
$$\theta_{t+1} = \theta_t - \eta v_{t+1}$$

Accelerates in relevant directions, dampens oscillations ($\beta \approx 0.9$)

3. Nesterov Momentum:

$$v_{t+1} = \beta v_t + \nabla_\theta \mathcal{L}(\theta_t - \beta v_t)$$
$$\theta_{t+1} = \theta_t - \eta v_{t+1}$$

Looks ahead before computing gradient

Adaptive Learning Rate Methods

4. **AdaGrad:** Adapts learning rate per parameter

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \nabla_\theta \mathcal{L}(\theta_t)$$

where $G_t = \sum_{i=1}^{t} (\nabla_\theta \mathcal{L}(\theta_i))^2$
Problem: Learning rate decreases too aggressively

5. **RMSprop:** Exponential moving average

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1-\beta)(\nabla_\theta \mathcal{L}(\theta_t))^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} \nabla_\theta \mathcal{L}(\theta_t)$$

6. **Adam (Adaptive Moment Estimation):** Most popular

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) \nabla_\theta \mathcal{L}(\theta_t)$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_\theta \mathcal{L}(\theta_t))^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

Typical: $\beta_1 = 0.9, \beta_2 = 0.999, \eta = 0.001$